

Review Questions

The answers to the chapter review questions can be found in the Appendix.

1. Which code can be inserted to have the code print 2?

```
public class BirdSeed {  
    private int numberBags;  
    boolean call;  
  
    public BirdSeed() {  
        // LINE 1  
        call = false;  
        // LINE 2  
    }  
  
    public BirdSeed(int numberBags) {  
        this.numberBags = numberBags;  
    }  
  
    public static void main(String[] args) {  
        var seed = new BirdSeed();  
        System.out.print(seed.numberBags);  
    } }
```

Es la E porque this se usa para llamar al constructor al poner this(2) se llama al constructor y se le asigna a this.numberBags = 2

- A. Replace line 1 with `BirdSeed(2);`.
 - B. Replace line 2 with `BirdSeed(2);`.
 - C. Replace line 1 with `new BirdSeed(2);`.
 - D. Replace line 2 with `new BirdSeed(2);`.
 - E. Replace line 1 with `this(2);`.
 - F. Replace line 2 with `this(2);`.
 - G. The code prints 2 without any changes.
2. Which modifier pairs can be used together in a method declaration? (Choose all that apply.)

- A. `static` and `final`
- B. `private` and `static`
- C. `static` and `abstract`
- D. `private` and `abstract`
- E. `abstract` and `final`
- F. `private` and `final`

A porque el metodo estatico puede ser final para evitar que se oculte en subclases
B un metodo privado puede ser estatico
F aunque sea redundante que sea privado y final ya que al ser privado ya no puede ser modificado

3. Which of the following statements about methods are true? (Choose all that apply.)

- A. Overloaded methods must have the same signature.
- B. Overridden methods must have the same signature.
- C. Hidden methods must have the same signature.
- D. Overloaded methods must have the same return type.
- E. Overridden methods must have the same return type.
- F. Hidden methods must have the same return type.

Los metodos sobrescritos deben tener la misma firma, mismo nombre y los mismos parametros.
Los metodos estaticos deben tener la misma firma.

4. What is the output of the following program?

```
1: class Mammal {  
2:     private void sneeze() {}  
3:     public Mammal(int age) {  
4:         System.out.print("Mammal");  
5:     } }  
6: public class Platypus extends Mammal {  
7:     int sneeze() { return 1; }  
8:     public Platypus() {  
9:         System.out.print("Platypus");  
10:    }  
11:    public static void main(String[] args) {  
12:        new Mammal(5);  
13:    } }
```

- A. Platypus
- B. Mammal
- C. PlatypusMammal
- D. MammalPlatypus
- E. The code will compile if line 7 is changed.
- F. The code will compile if line 9 is changed.

La clase Mammal no tiene un constructor sin argumentos y Java intenta insertar super en el constructor de Platypus pero como el constructor de Mammal no esta vacio entonces no compila

5. Which of the following complete the constructor so that this code prints out 50? (Choose all that apply.)

```
class Speedster {  
    int numSpots;  
}  
public class Cheetah extends Speedster {  
    int numSpots;  
  
    public Cheetah(int numSpots) {  
        // INSERT CODE HERE  
    }  
}
```

```

    public static void main(String[] args) {
        Speedster s = new Cheetah(50);
        System.out.print(s.numSpots);
    }
}

```

E es correcta porque se le asigna 50 a Speedster

- A. numSpots = numSpots;
- B. numSpots = this.numSpots;
- C. this.numSpots = numSpots;
- D. numSpots = super.numSpots;
- E. super.numSpots = numSpots;**
- F. The code does not compile regardless of the code inserted into the constructor.
- G. None of the above

6. Which of the following declare immutable classes? (Choose all that apply.)

```

public final class Moose {
    private final int antlers;
}

```

```

public class Caribou {
    private int antlers = 10;
}

```

```

public class Reindeer {
    private final int antlers = 5;
}

```

```

public final class Elk {}

```

```

public final class Deer {
    private final Object o = new Object();
}

```

- A. Moose
- B. Caribou
- C. Reindeer
- D. Elk**
- E. Deer**
- F. None of the above

Elk es immutable ya que no tiene ni variables asignadas digamos no tiene nada que modificar

Deer ya que al poner final Object ya no puede ser modificado

7. What is the output of the following code?

```
1: class Arthropod {
2:     protected void printName(long input) {
3:         System.out.print("Arthropod");
4:     }
5:     void printName(int input) {
6:         System.out.print("Spooky");
7:     } }
8: public class Spider extends Arthropod {
9:     protected void printName(int input) {
10:        System.out.print("Spider");
11:    }
12:    public static void main(String[] args) {
13:        Arthropod a = new Spider();
14:        a.printName((short)4);
15:        a.printName(4);
16:        a.printName(5L);
17:    } }
```

- A. SpiderSpiderArthropod
- B. SpiderSpiderSpider
- C. SpiderSpookyArthropod
- D. SpookySpiderArthropod
- E. The code will not compile because of line 5.
- F. The code will not compile because of line 9.
- G. None of the above

8. What is the result of the following code?

```
1: abstract class Bird {
2:     private final void fly() { System.out.println("Bird"); }
3:     protected Bird() { System.out.print("Wow-"); }
4: }
5: public class Pelican extends Bird {
6:     public Pelican() { System.out.print("Oh-"); }
7:     protected void fly() { System.out.println("Pelican"); }
8:     public static void main(String[] args) {
9:         var chirp = new Pelican();
10:        chirp.fly();
11:    } }
```

Imprime Wow Oh Pelican porque al hacer new Pelican automaticamente se pone el super, despues Pelican imprime Oh y al final al hacer chip.fly imprime Pelican ya que el metodo fly que llama es el propio del Pelican porque el metodo fly de Bird es privado entonces no es heredado.

- A. Oh-Bird
 - B. Oh-Pelican
 - C. Wow-Oh-Bird
 - D. Wow-Oh-Pelican**
 - E. The code contains a compilation error.
 - F. None of the above
9. Which of the following statements about overridden methods are true? (Choose all that apply.)
- A. An overridden method must contain method parameters that are the same or covariant with the method parameters in the inherited method.
 - B. An overridden method may declare a new exception, provided it is not checked.**
 - C. An overridden method must be more accessible than the method in the parent class.
 - D. An overridden method may declare a broader checked exception than the method in the parent class.
 - E. If an inherited method returns void, then the overridden version of the method must return void.**
 - F. None of the above

10. Which of the following pairs, when inserted into the blanks, allow the code to compile? (Choose all that apply.)

```

1: public class Howler {
2:     public Howler(long shadow) {
3:         _____;
4:     }
5:     private Howler(int moon) {
6:         super();
7:     }
8: }
9: class Wolf extends Howler {
10:     protected Wolf(String stars) {
11:         super(2L);
12:     }
13:     public Wolf() {
14:         _____;
15:     }
16: }
```

- A. this(3) at line 3, this("") at line 14**
- B. this() at line 3, super(1) at line 14
- C. this((short)1) at line 3, this(null) at line 14**
- D. super() at line 3, super() at line 14

B porque se pueden agregar excepciones pero solo que sean unchecked.
E porque tiene que retornar void ya que si es lo que retorna el padre tambien eso tiene que retornar el hijo

A compila porque this(3) accede a los constructores private de la misma clase y this("") es valido porque la cadena llega al constructor.
C compila porque short sube a int y encuentra a Howler(int) y this(null) es compatible con String.

- E. `this(2L)` at line 3, `super((short)2)` at line 14
- F. `this(5)` at line 3, `super(null)` at line 14
- G. Remove lines 3 and 14.

11. What is the result of the following?

```

1: public class PolarBear {
2:     StringBuilder value = new StringBuilder("t");
3:     { value.append("a"); }
4:     { value.append("c"); }
5:     private PolarBear() {
6:         value.append("b");
7:     }
8:     public PolarBear(String s) {
9:         this();
10:        value.append(s);
11:    }
12:    public PolarBear(CharSequence p) {
13:        value.append(p);
14:    }
15:    public static void main(String[] args) {
16:        Object bear = new PolarBear();
17:        bear = new PolarBear("f");
18:        System.out.println(((PolarBear)bear).value);
19:    } }
```

- A. `tacb`
- B. `tacf`
- C. `tacbf`
- D. `tcafb`
- E. `taftacb`
- F. The code does not compile.
- G. An exception is thrown.

El primer objeto pasa por los bloques y el constructor pero despues se crea un nuevo objeto y el objeto anterior queda descartado, entonces se imprime el nuevo objeto que seria `tacbf`

12. How many lines of the following program contain a compilation error?

```

1: public class Rodent {
2:     public Rodent(Integer x) {}
3:     protected static Integer chew() throws Exception {
4:         System.out.println("Rodent is chewing");
5:         return 1;
6:     }
```

```

7:  }
8:  class Beaver extends Rodent {
9:      public Number chew() throws RuntimeException {
10:          System.out.println("Beaver is chewing on wood");
11:          return 2;
12:      } }

```

En la línea 8 está el error porque el constructor de Rodent está vacío
Y en public Number chew genera un error ya que intenta sobrescribir un método static con uno de instancia.

- A. None
- B. 1
- C. 2**
- D. 3
- E. 4
- F. 5

13. Which of these classes compile and will include a default constructor created by the compiler? (Choose all that apply.)

A.

```
public class Bird {}
```

B.

```
public class Bird {
    public bird() {}
}
```

A porque sin constructores Java crea Bird por defecto entonces compila

C.

```
public class Bird {
    public bird(String name) {}
}
```

bird es un método no es un constructor

D.

```
public class Bird {
    public Bird() {}
}
```

E.

```
public class Bird {
    Bird(String name) {}
}
```

F.

```
public class Bird {
    private Bird(int age) {}
}
```

G.

```
public class Bird {
    public Bird bird() { return null; }
}
```

14. Which of the following statements about inheritance are correct? (Choose all that apply.)

- A.** A class can directly extend any number of classes.
- B.** A class can implement any number of interfaces.
- C.** All variables inherit `java.lang.Object`.
- D.** If class A is extended by B, then B is a superclass of A.
- E.** If class C implements interface D, then C is a subtype of D.
- F.** Multiple inheritance is the property of a class to have multiple direct superclasses.

15. Which statements about the following program are correct? (Choose all that apply.)

```
1: abstract class Nocturnal {
2:     boolean isBlind();
3: }
4: public class Owl extends Nocturnal {
5:     public boolean isBlind() { return false; }
6:     public static void main(String[] args) {
7:         var nocturnal = (Nocturnal)new Owl();
8:         System.out.println(nocturnal.isBlind());
9:     } }
```

- A.** It compiles and prints true.
- B.** It compiles and prints false.
- C.** The code will not compile because of line 2.
- D.** The code will not compile because of line 5.
- E.** The code will not compile because of line 7.
- F.** The code will not compile because of line 8.
- G.** None of the above

La línea 2 es `boolean isBlind()` dentro de una clase abstract. En Java, los métodos abstractos solo pueden existir en clases abstract o interfaces. Pero el problema es que falta la palabra clave `abstract` antes del método. Lo correcto sería `abstract boolean isBlind()`

16. What is the result of the following?

```
1: class Arachnid {
2:     static StringBuilder sb = new StringBuilder();
3:     { sb.append("c"); }
4:     static
5:     { sb.append("u"); }
6:     { sb.append("r"); }
7: }
```



```

8: public class Scorpion extends Arachnid {
9:     static
10:    { sb.append("q"); }
11:    { sb.append("m"); }
12:    public static void main(String[] args) {
13:        System.out.print(Scorpion.sb + " ");
14:        System.out.print(Scorpion.sb + " ");
15:        new Arachnid();
16:        new Scorpion();
17:        System.out.print(Scorpion.sb);
18:    } }

```

- A. qu qu qumrcrc
 - B. u u ucrcrm
 - C. uq uq uqmcrcr
 - D. uq uq uqcrCRM**
 - E. qu qu qumrcrc
 - F. qu qu qucrCRM
 - G. The code does not compile.
17. Which of the following are true? (Choose all that apply.)
- A. `this()` can be called from anywhere in a constructor.
 - B. `this()` can be called from anywhere in an instance method.
 - C. `this.variableName` can be called from any instance method in the class.**
 - D. `this.variableName` can be called from any static method in the class.
 - E. You can call the default constructor written by the compiler using `this()`.
 - F. You can access a private constructor with the `main()` method in the same class.**
18. Which statements about the following classes are correct? (Choose all that apply.)
- ```

1: public class Mammal {
2: private void eat() {}
3: protected static void drink() {}
4: public Integer dance(String p) { return null; }
5: }
6: class Primate extends Mammal {
7: public void eat(String p) {}
8: }
9: class Monkey extends Primate {

```

C porque `this.variable` es válido en cualquier método de instancia para referenciar campos del objeto actual  
`main()` está dentro de la misma clase, por lo que puede acceder a miembros `private`

```

10: public static void drink() throws RuntimeException {}
11: public Number dance(CharSequence p) { return null; }
12: public int eat(String p) {}
13: }

```

- A. The eat() method in Mammal is correctly overridden on line 7.
- B. The eat() method in Mammal is correctly overloaded on line 7.
- C. The drink() method in Mammal is correctly overridden on line 10.
- D. The drink() method in Mammal is correctly hidden on line 10.**
- E. The dance() method in Mammal is correctly overridden on line 11.
- F. The dance() method in Mammal is correctly overloaded on line 11.**
- G. The eat() method in Primate is correctly hidden on line 12.
- H. The eat() method in Primate is correctly overloaded on line 12.

D porque Monkey declara static void drink(), ocultando correctamente el static de Mammal. F porque dance(CharSequence) y dance(String) son métodos diferentes entonces la sobrecarga es válida.

19. What is the output of the following code?

```

1: class Reptile {
2: {System.out.print("A");}
3: public Reptile(int hatch) {}
4: void layEggs() {
5: System.out.print("Reptile");
6: } }
7: public class Lizard extends Reptile {
8: static {System.out.print("B");}
9: public Lizard(int hatch) {}
10: public final void layEggs() {
11: System.out.print("Lizard");
12: }
13: public static void main(String[] args) {
14: var reptile = new Lizard(1);
15: reptile.layEggs();
16: } }

```

- A. AALizard
- B. BALizard
- C. BLizardA
- D. ALizard
- E. The code will not compile because of line 3.
- F. None of the above**

La clase Reptile define un constructor, pero no es un constructor sin argumentos. Por lo tanto, el constructor de Lizard debe llamar explícitamente a super, pasando un valor de tipo int por esto no compila.

20. Which statement about the following program is correct?

```
1: class Bird {
2: int feathers = 0;
3: Bird(int x) { this.feathers = x; }
4: Bird fly() {
5: return new Bird(1);
6: } }
7: class Parrot extends Bird {
8: protected Parrot(int y) { super(y); }
9: protected Parrot fly() {
10: return new Parrot(2);
11: } }
12: public class Macaw extends Parrot {
13: public Macaw(int z) { super(z); }
14: public Macaw fly() {
15: return new Macaw(3);
16: }
17: public static void main(String... sing) {
18: Bird p = new Macaw(4);
19: System.out.print(((Parrot)p.fly()).feathers);
20: } }
```

- A. One line contains a compiler error.
  - B. Two lines contain compiler errors.
  - C. Three lines contain compiler errors.
  - D. The code compiles but throws a `ClassCastException` at runtime.
  - E. The program compiles and prints 3.
  - F. The program compiles and prints 0.
21. Which of the following are properties of immutable classes? (Choose all that apply.)
- A. The class can contain setter methods, provided they are marked `final`.
  - B. The class must not be able to be extended outside the class declaration.
  - C. The class may not contain any instance variables.
  - D. The class must be marked `static`.
  - E. The class may not contain any `static` variables.
  - F. The class may only contain `private` constructors.
  - G. The data for mutable instance variables may be read, provided they cannot be modified by the caller.

B :Marcando la clase como `final` evitas que alguien la extienda y agregue estado mutable.

C: Puedes tener getters para leer el estado, siempre que no haya forma de modificarlo desde fuera.

22. What does the following program print?

```
1: class Person {
2: static String name;
3: void setName(String q) { name = q; } }
4: public class Child extends Person {
5: static String name;
6: void setName(String w) { name = w; }
7: public static void main(String[] p) {
8: final Child m = new Child();
9: final Person t = m;
10: m.name = "Elysia";
11: t.name = "Sophia";
12: m.setName("Webby");
13: t.setName("Olivia");
14: System.out.println(m.name + " " + t.name);
15: } }
```

A. Elysia Sophia

B. Webby Olivia

C. Olivia Olivia

D. Olivia Sophia

E. The code does not compile.

F. None of the above

hay dos variables name separadas:

Child.name termina valiendo "Olivia"

Person.name termina valiendo "Sophia"

23. What is the output of the following program?

```
1: class Canine {
2: public Canine(boolean t) { logger.append("a"); }
3: public Canine() { logger.append("q"); }
4:
5: private StringBuilder logger = new StringBuilder();
6: protected void print(String v) { logger.append(v); }
7: protected String view() { return logger.toString(); }
8: }
9:
10: class Fox extends Canine {
11: public Fox(long x) { print("p"); }
12: public Fox(String name) {
13: this(2);
14: print("z");
15: }
```

```

16: }
17:
18: public class Fennec extends Fox {
19: public Fennec(int e) {
20: super("tails");
21: print("j");
22: }
23: public Fennec(short f) {
24: super("eevee");
25: print("m");
26: }
27:
28: public static void main(String... unused) {
29: System.out.println(new Fennec(1).view());
30: } }

```

A. qpz

**B. qpzj**

C. jzpa

D. apj

E. apjm

F. The code does not compile.

G. None of the above

B porque la cadena se forma porque los constructores se ejecutan de adentro hacia afuera Canine agrega q, Fox(long) agrega p, Fox(String) agrega z y Fennec agrega j.

**24.** What is printed by the following program?

```

1: class Antelope {
2: public Antelope(int p) {
3: System.out.print("4");
4: }
5: { System.out.print("2"); }
6: static { System.out.print("1"); }
7: }
8: public class Gazelle extends Antelope {
9: public Gazelle(int p) {
10: super(6);
11: System.out.print("3");
12: }

```

```
13: public static void main(String hopping[]) {
14: new Gazelle(0);
15: }
16: static { System.out.print("8"); }
17: { System.out.print("9"); }
18: }
```

- A. 182640
- B. 182943
- C. 182493
- D. 421389
- E. The code does not compile.
- F. The output cannot be determined until runtime.

25. Which of the following are true about a concrete class? (Choose all that apply.)

- A. A concrete class can be declared as `abstract`.
- B. A concrete class must implement all inherited abstract methods.
- C. A concrete class can be marked as `final`.
- D. A concrete class must be immutable.
- E. A concrete method that implements an abstract method must match the method declaration of the abstract method exactly.

Si no implementa todos los métodos abstract, debe declararse `abstract` ella misma. Una clase concreta puede ser `final` para evitar que se extienda

26. What is the output of the following code?

```
4: public abstract class Whale {
5: public abstract void dive();
6: public static void main(String[] args) {
7: Whale whale = new Orca();
8: whale.dive(3);
9: }
10: }
11: class Orca extends Whale {
12: static public int MAX = 3;
13: public void dive() {
14: System.out.println("Orca diving");
15: }
16: public void dive(int... depth) {
17: System.out.println("Orca diving deeper " + MAX);
18: } }
```

- A. Orca diving
- B. Orca diving deeper 3
- C. The code will not compile because of line 4.
- D. The code will not compile because of line 8.**
- E. The code will not compile because of line 11.
- F. The code will not compile because of line 12.
- G. The code will not compile because of line 17.
- H. None of the above

whale.dive(3) intenta llamar a un método dive con argumento int. La referencia whale es tipo Whale (abstract).  
Whale solo tiene: public abstract void dive() sin parámetros.  
El compilador busca dive(int) en Whale y no lo encuentra  
Error de compilación